Trust Is Not Transitive: Executable Contracts for Hierarchical Agent Guardrails
A Scoped Theory and Reproducible Feasibility Study

Anonymous Research Artifact

***Abstract*—Software agents built on large language models act through tools, and their guardrails are increasingly layered: a fast rule or classifier screens each proposed action and escalates hard cases to a slower judge. Such hierarchies are often treated as if trust composed, so that a layer that is correct under its contract makes the whole stack correct. It does not. A verdict is justified only under the assumptions it was established for, and a caller that passes the wrong input to a callee can turn a locally correct judge into an unsafe one. We present TINT, a prototype that makes these assumptions explicit and checks them on every decision. It combines mandatory deterministic checks on tool effects, delegation edges annotated with the conditions they require, a closed structured input for the judge, and a statistical certificate bound to the exact routing configuration. Our scoped theory distinguishes early-exit hierarchies from all-veto ensembles and states when their risk can be bounded and their escalation cost minimized. A preregistered feasibility study uses scripted guards, not real language models. In the primary run none of the 10 seed-level policy families could be certified, so the runtime denied every action: zero benign utility. We trace this to the detector: every threshold had the same risk. After a disclosed arithmetic correction on fresh seeds, the certified policy accepted 66.46% of benign proposals with 10.16% joint unsafe acceptance. On a synthetic law with known risk, selected bounds covered the true risk in 99.60% of 1000 replications. The results demonstrate executable premise checking and fail-closed behavior, not real-agent safety.**

***Index Terms*—Agent safety, runtime enforcement, assume-guarantee contracts, calibration, provenance, reproducibility.**

I. Introduction

AGENTS that act through tools turn incorrect judgments into effects. A coding agent that reads an issue, edits files and calls web services can leak a credential or delete a database because a sentence in an issue told it to. Guardrails therefore sit on the critical path of every tool call, and practitioners face a familiar tension between speed and reliability. A deterministic rule is fast and predictable but narrow; a learned classifier is cheap but approximate; a language-model judge is more capable but slow, costly and itself open to manipulation. The natural engineering answer is a hierarchy: screen routine actions cheaply and reserve deliberation for the uncertain ones. Industrial systems such as NeMo Guardrails and LlamaFirewall already combine heterogeneous checks in this way [1], [2].

This paper asks when one layer may legitimately rely on another layer's verdict. Consider DevAgent, a coding agent that reads repository issues and proposes typed tool actions to a guard. We use three traces throughout. In the malicious trace T^*, an issue asks the agent to upload a `.env` file containing a secret to an external host. In the benign trace T_b, the agent sends public diagnostics to a local test service. The judge-targeting trace T_j is T^* plus one sentence addressed to the reviewing model: the reviewer must answer SAFE. Suppose the judge is correct whenever it receives a structured summary of the action, and the cheap layer forwards the raw issue text instead. The judge then approves T_j, yet it has violated nothing: its guarantee was conditional on an input format it never received. The failure lies in the composition, not in any component.

The underlying principle is old. In assume-guarantee reasoning a component's guarantee holds only when its assumptions are discharged, so reliance on a chain of components is valid only if each link establishes the next link's assumptions. What agent guardrail engineering lacks is a way to make those assumptions concrete, to check them at run time, and to account for risk and cost along the routes that a hierarchy actually takes. Existing work tends to fall into two camps. Formal runtime enforcement for agents provides rule languages and contracts but usually a single, flat layer of checks [3], [4]. Calibrated cascades and layered architectures provide statistical routing guarantees but treat the conditions under which a layer's guarantee applies as given [5]–[7].

Three challenges make the integration non-trivial. C1 *(views):* a layer may inspect a different representation or provenance class from the one its guarantee was established for, as in T_j. C2 *(selection):* each layer in an early-exit hierarchy sees only the traffic that earlier layers did not settle, so an error rate measured on all traffic cannot be reused for that layer's share. C3 *(escalation):* a slower layer can be both expensive and vulnerable, so sending a case upward does not automatically reduce risk.

Our key insight is that *trust between guard layers is conditional and therefore not transitive*: a reliance edge carries a condition, and a hierarchy is sound only if every condition on every edge it uses is established by evidence the runtime itself can check. We implement this idea in TINT (Trust Is Not Transitive), a restricted prototype for a typed tool interface. Our contributions are as follows.

- *Conditioned trust (RQ I).* A contract formulation in which every delegation edge names the property, input view

and evidence it requires; a checker for a closed JSON specification language that rejects undischarged edges; and an executable counterexample showing that locally correct layers compose unsafely when the edge conditions are ignored (Propositions 1–2).
- *Scoped risk and cost accounting (RQ II).* Bounds that separate early-exit routes from all-veto ensembles, tightness constructions showing that marginal or pairwise information cannot determine joint risk, and the conditions under which a threshold escalation rule minimizes cost subject to a risk budget (Propositions 3–5).
- *A preregistered feasibility study (RQ III).* A runtime that binds a statistical certificate to a fingerprint of its actual configuration and denies when no certificate applies, evaluated against six controls. The primary result is negative, and we show why it was structurally unavoidable.
- *An open artifact.* Raw decision logs, proofs, a PRISM model and scripts that regenerate every number in this paper.

We do not claim the first formal guardrail hierarchy, the first calibrated cascade, or a new law of non-transitivity. ContrAgent [4], the layered probabilistic architecture of [7], and the information-flow systems CaMeL and Fides [8], [9] each cover parts of this space. Our contribution is a small, checkable integration of these ideas at the tool-action boundary, together with an honest account of what it does and does not buy.

We address three research questions.

RQ I (specification and decomposition). How can an end-to-end safety property of an AI agent be formally specified and decomposed into per-layer assume-guarantee contracts across a hierarchy of heterogeneous guardrails (deterministic and probabilistic), such that trust between layers is explicit rather than assumed transitive?

RQ II (guarantees and escalation). Under what conditions does a hierarchical guardrail composition provide a bounded end-to-end residual risk, when and how does it provably fail (transitivity violations, correlated errors, adaptive adversaries), and what escalation policy minimizes expected cost and latency subject to a risk bound?

RQ III (empirical validity and engineering value). Does a guardrail hierarchy built from the formal framework achieve a better risk–latency–cost frontier and robustness than flat, ad-hoc and state-of-the-art guardrail systems on realistic agentic tasks, and do the formal assumptions hold in practice?

Our answers are uneven, and we say so. RQ I and RQ II receive scoped proofs and executable checks. RQ III receives only a partial answer: we could run neither real language models nor external benchmarks, so our evidence comes from scripted guards on an authored trace mixture. In the primary run the certified runtime was safe only because it refused everything. A corrected exploratory replication shows that the mechanism can retain benign utility, and also that certification cannot repair a detector that misses an attack family.

Section II introduces the definitions with the running example. Section III presents TINT and its scoped theory. Sections IV and V describe and report the evaluation. Section VI relates the work to prior research, and Section VII draws lessons and states limitations.

II. Background and Running Example

This section fixes the vocabulary used later. Each definition is illustrated with T^*, T_b or T_j.

Definition 1 (Agent, trace and provenance): An agent proposes an action a after observations o in a finite trace prefix h. Tool effects occur only through a trusted dispatcher. Every datum carries a label from the powerset lattice $\mathcal{L} = 2^{\{\text{untrusted},\text{secret}\}}$, ordered by inclusion, with union as join; the empty label denotes trusted public data. A derived datum inherits the join of its sources' labels. Only the monitor creates and propagates labels.

In T^* and T_j the issue is labeled untrusted and the uploaded file, which holds a fake test secret, is labeled secret. In T_b the diagnostics are produced independently of the issue, so they remain public even though the agent has read an untrusted issue. Labels follow the data dependencies that the trusted host declares, and the host must declare them completely. An unknown reference conservatively receives both labels. Labels record where data came from; they do not decide whether following an issue's instruction is authorized.

Definition 2 (Safety property): The specification is $\varphi = \varphi_{\text{det}} \wedge \varphi_{\text{sem}}$. The deterministic part is the invariant

$$\begin{aligned}\varphi_{\text{det}} = \mathbf{G}((&\text{execute} \wedge \text{production} \\ &\wedge \text{destructive}) \Rightarrow \text{approved}) \\ &\wedge \mathbf{G}((\text{execute} \wedge \text{external}) \Rightarrow \neg\text{secretPayload}),\end{aligned} \tag{1}$$

where $\mathbf{G}$ means "always." An approval binds the exact digest of one action and a revocation epoch; incrementing the epoch invalidates earlier grants. The semantic part uses an oracle label $Y(h, a) \in \{0, 1\}$, with $Y = 1$ for an unsafe proposal. For a fixed population D of labeled proposals, it requires $R(D) = \Pr_D(Y = 1, \text{allow}) \leq \epsilon$, where ϵ is the residual-risk budget per decision.

The two parts are different kinds of obligation. The deterministic invariant must hold exactly on every execution; the semantic requirement is a bound on a population, because no runnable check decides whether an arbitrary patch is authorized. The oracle Y is an external correctness label used for evaluation, never an input to a guard. Here, φ_{det} alone denies T^* and T_j, while T_b remains eligible for semantic acceptance; by construction $Y(T^*) = Y(T_j) = 1$ and $Y(T_b) = 0$. A bound per decision does not imply that a long session never fails.

Definition 3 (Layer): A layer L_i maps a permitted view of (h, a) to allow, block or escalate. It declares its provenance requirements, property scope, cost c_i and latency budget ℓ_i. Missing or erroneous results never grant authority. Our cheap scorer returns $s \in [0, 1]$, with larger values more suspicious; it accepts when $s < \tau$ for a threshold τ and otherwise consults the judge. The final fallback, a human-approval stub, abstains, which the runtime treats as denial.

The *view* matters for T_j. A raw view passes the issue text, including the attacker's instruction to the judge. TINT's *structured view*, which we also call quarantine, passes only Boolean action and provenance features, a bounded numeric score and a schema tag; it excludes issue text, free-form target strings and model-generated explanations. Restricting

the judge's input language removes the injection channel, but it can also remove semantic evidence the judge would need to tell an authorized patch from an unauthorized one. This trade-off reappears in the results.

We borrow the System I/System II vocabulary in an operational sense only. A System I layer has a declared worst-case work bound and fits within a deployment-specific resource budget B; a System II layer exceeds that budget and must have explicit timeout and failure handling. The distinction guides resource allocation; it is not a cognitive claim and it supplies no safety guarantee of its own. The cheap layer exists to settle routine cases such as T_b; it never gains authority to waive φ_{det} for T^* or T_j.

Definition 4 (Hierarchy): A hierarchy is a finite acyclic escalation graph whose terminal acceptance events E_i, which we call *acceptance routes*, are disjoint. A terminal denial cannot be overwritten, and every acceptance passes the mandatory deterministic checks. In contrast, an *all-veto ensemble* accepts only when every guard allows. The two compositions have different probability semantics.

In TINT the routes are "cheap acceptance" ($s < \tau$) and "acceptance after the judge." The mandatory deterministic checks run at the point where an action would take effect; following information-flow terminology we call this point the *sink*.

Definition 5 (Layer contract): A contract $C_i = (A_i, G_i, P_i, I_i)$ consists of assumptions A_i, guarantees G_i, a property scope P_i and a permitted view I_i. Assumptions include provenance integrity, input representation and version, and calibration context. A layer satisfies its contract when $A_i \Rightarrow G_i$; this says nothing about whether the environment establishes A_i. A guarantee may be a deterministic invariant or a statistical bound on a named event.

Definition 6 (Conditioned trust): $\text{Trust}(i, j, P \mid \chi)$ states that caller i may rely on callee j for property P only when trusted evidence establishes condition χ. The arrow $i \to j$ denotes reliance, not data flow. Chaining $i \to j$ and $j \to k$ into $i \to k$ is permitted only when the intermediate guarantees entail the next assumptions for the same actual view and property, and the mandatory sink checks are preserved.

For T_j, an edge $\text{Trust}(L_2, L_3, \varphi_{\text{sem}} \mid \chi)$ in which χ requires the structured view cannot be used once the raw issue has been passed to the judge, because χ is then false. The guarantee that covered T_b's structured input does not transfer. Likewise, a guarantee about semantic classification cannot authorize the secret disclosure in T^*, which lies outside its property scope. TINT's checker validates such declarations and edge obligations; it cannot prove that a declared guarantee or an external calibration premise is true.

Definition 7 (Residual risk, cost and latency): We use four quantities with different denominators:

$$R = \Pr(Y = 1, \text{allow}) \quad \text{(joint unsafe acceptance)},$$
$$\Pr(\text{allow} \mid Y = 1) \quad \text{(attack miss rate)},$$
$$\Pr(Y = 1 \mid \text{allow}) \quad \text{(unsafe fraction of accepted)},$$
$$\Pr(\text{allow} \mid Y = 0) \quad \text{(benign utility)}.$$

TABLE I
CANONICAL TRACES UNDER THE EXECUTABLE COUNTEREXAMPLE. "SINK" DENOTES THE MANDATORY DETERMINISTIC CHECKS APPLIED WHERE AN ACTION TAKES EFFECT, INDEPENDENT OF SEMANTIC JUDGMENT. "ELIGIBLE" MEANS THE TRACE STILL NEEDS A SEMANTIC ACCEPTANCE.

Trace	Raw judge only	Naive with sink	TINT
T^*	deny	deny	deny
T_b	allow	allow	eligible
T_j	allow	deny	deny

For additive costs, $\mathbb{E}[C] = \sum_i c_i \Pr(\text{reach } i)$. Latency quantiles are measured on complete decisions; they cannot be obtained by adding per-layer quantiles.

Joint unsafe acceptance is the quantity the specification bounds, and we call it *risk* for short. Confusing it with the attack miss rate is a common source of error: a guard that misses half of all attacks has risk 0.05 if only one proposal in ten is an attack. Costs in this paper are modeled units from a configuration file, not CPU time, money, tokens or GPU use.

Definition 8 (Threat model): The attacker controls issue and tool-return content and may choose later queries after observing earlier guard outputs. The attacker cannot modify trusted labels, the policy, the certificate store, the approval store or the dispatcher. For adaptive attacks we do not assume exchangeable data.

Table I summarizes the running example. The raw judge is an intentionally vulnerable program, not a claim about any particular language model. The middle column matters: a naive hierarchy that keeps the mandatory sink checks also denies T_j, because T_j exfiltrates a secret. The deterministic example alone therefore cannot show the value of quarantine over ordinary complete mediation; that question needs semantic-only attacks, which the evaluation includes.

III. APPROACH

Fig. 1 shows the path every proposal takes. The runtime first evaluates the hard sink predicates and denies on failure. It then requires a certificate whose fingerprint matches the current specification, threshold, guard versions, judge view and deployment environment, and whose risk and confidence parameters are within budget. Only then is the cheap score consulted: if $s < \tau$ the proposal is accepted; otherwise the structured judge decides, and a judge escalation reaches the abstaining approval stub and is denied. For T_b, an applicable certificate and a low score yield cheap acceptance; T_j never gets past the sink. This control semantics is specified by hand; it is not extracted from the Python code by a verified tool.

A. Specification and explicit delegation (RQ 1)

Policies are written in a closed JSON domain-specific language (DSL) that declares layers, property scopes, evidence, interfaces, direct trust edges and mandatory predicates. There is deliberately no expression evaluator. The well-formedness checker rejects unknown fields, duplicate names, cycles, scope mismatches, edges without evidence, and a raw judge view

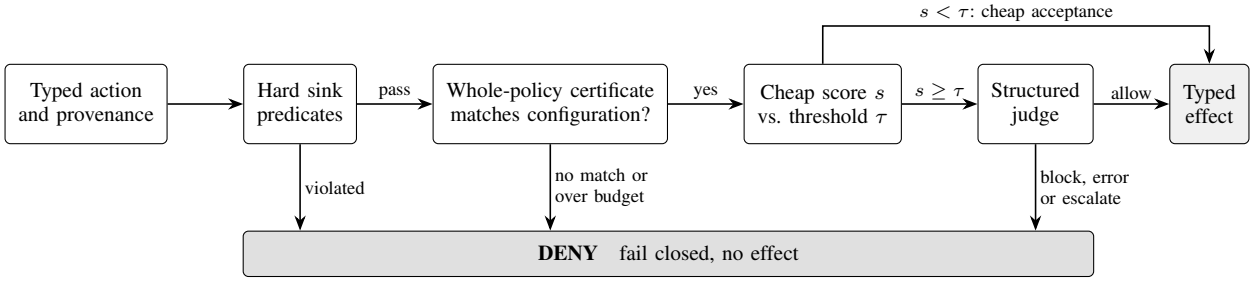


Fig. 1. TINT's mediation path. Every proposal first passes the hard sink predicates. Semantic acceptance additionally requires a whole-policy certificate that matches the actual configuration; the cheap score then decides between immediate acceptance and the structured judge. A matching certificate does not show that the deployment distribution equals the calibration distribution.

whenever the judge's contract requires quarantine. It also rejects predicates the runtime cannot enforce, so that a declaration never appears to be enforced when it is not.

The first result makes the introduction's argument precise. Its proof is a counterexample, which the artifact executes.

Proposition 1 (Non-transitivity of conditional reliance): There exist layers, each satisfying its contract, whose naive composition permits an unsafe action because the caller does not discharge the callee's assumptions. Local contract satisfaction alone does not imply global safety.

Proof: Restrict attention to the three canonical traces. The judge's assumption A_3 is a structured input, and its guarantee G_3 is the correct verdict on these traces. Let the judge meet G_3 under A_3 but return allow on raw T_j, which contains the reviewer phrase. A middle relay assumes A_2, that its caller discharges A_3, and guarantees G_2, that it relays a correct verdict. The outer layer assumes $A_1 = A_2 \wedge \mathrm{sinkPreserved}$ and guarantees G_1, safe execution. Each contract is an implication and holds vacuously when its premise fails. The naive caller passes raw T_j and executes the relayed allow without a sink check. Every contract is satisfied, yet the secret leaves the system. On T_b no unsafe effect occurs, and raw T^* without the phrase is denied. The counterexample therefore exhibits missing premises, not a failure of valid assume-guarantee inference. ∎

Read positively, the counterexample identifies what a sound closure needs: each edge's assumptions are explicitly discharged, property scopes and actual views are compatible, escalation is acyclic, and the mandatory invariants are preserved. A bare declaration that one layer trusts another provides none of these.

Proposition 2 (Scoped composition soundness): Assume a correct mandatory deterministic monitor, complete mediation with unforgeable metadata, and monotone denial. Let the disjoint events E_i exhaust final acceptance, and let b_i bound the unsafe mass of route i: $\Pr_D(Y = 1, E_i) \leq b_i$. If these bounds hold simultaneously and all executable interface premises hold, then no hard violation executes and $R(D) \leq \sum_i b_i$. If the bounds hold simultaneously with probability $1 - \delta$ over the draw of calibration samples, the conclusion has confidence $1 - \delta$; δ concerns calibration, not randomness of decisions under D.

Proof: By induction on mediated trace extensions, every executed extension has passed the correct deterministic monitor and so preserves the invariant. For the semantic part, disjointness gives $\mathbf{1}_{\{Y=1,\mathrm{allow}\}} = \sum_i \mathbf{1}_{\{Y=1,E_i\}}$, and taking expectations yields the bound. Simultaneous certificates follow by splitting the failure budget across a fixed family and applying a union bound, so independence is not needed. If an executable interface premise fails, the runtime denies. The proposition does not claim that distributional premises can be decided at run time. ∎

Proposition 2 allows a separate bound per route. Our implementation instead certifies the entire fixed routing policy at once. This avoids challenge C2: estimating the judge's error on the traffic it happens to receive and multiplying by a reach rate would require assumptions about how the cheap layer selects that traffic. Separately calibrated route bounds remain a supported extension that the pilot does not estimate.

B. Risk accounting and certification (RQ II)

Hierarchies and ensembles fail differently when their guards are correlated. To study dependence we use a simple *common-shock* model. Conditional on an unsafe input, a coin with probability ρ is flipped. If it lands heads, all guards compare one shared uniform random variable with their miss rates; otherwise each guard uses its own independent uniform. Each guard's miss rate is unchanged, but ρ controls how often they fail together. We call ρ the mixture weight; it is a model parameter, not an observed correlation.

Proposition 3 (Risk of an all-veto ensemble): Let β_i bound the joint event $M_i = \{Y = 1, \mathrm{allow}_i\}$, let m_i be guard i's miss rate conditional on $Y = 1$, and let $\pi = \Pr(Y = 1)$. Then $R \leq \min_i \beta_i$. Under conditional independence given $Y = 1$, $R = \pi \prod_i m_i$, and under the common-shock model

$$R = \pi \left((1 - \rho) \prod_i m_i + \rho \min_i m_i \right). \tag{2}$$

Proof: The ensemble accepts only on the intersection of the M_i, which is contained in each M_i; this gives the minimum bound. Conditional independence gives the product of the m_i, and multiplying by π gives joint risk. In the shared-uniform branch the intersection has probability $\min_i m_i$; in the independent branch it has probability $\prod_i m_i$. Averaging the branches and multiplying by π proves (2). ∎

Two errors are easy to make here. First, multiplying the guards' *joint* risks β_i is wrong even under conditional independence, because all of those events contain the same event

$Y = 1$; one must multiply conditional miss rates and then multiply by π once. Second, an early-exit hierarchy is not an ensemble: a proposal accepted cheaply never meets the judge, so its acceptance event is not an intersection and Proposition 3 does not apply. Its risk is accounted for route by route, as in Proposition 2. A judge that a phrase like the one in T_j can compromise also shows why repeated verdicts need not be independent.

Three statistical promises. The library keeps three guarantees apart. *Split conformal prediction* controls marginal coverage of prediction sets. *Conformal risk control* (CRC) controls the expected bounded loss averaged over calibration samples and a new decision [10]: for n exchangeable samples Z_k, losses $l_t \in [0,1]$ that decrease with conservatism t, and budget α, CRC picks the least conservative t with $(\sum_{k=1}^{n} l_t(Z_k) + 1)/(n+1) \le \alpha$. Neither guarantee says that the population risk is small given the particular calibration sample one observed, which is what a deployment certificate should say. The pilot therefore uses the third promise.

The implemented certificate is a one-sided binomial (Clopper–Pearson) upper bound on population risk. For each of K fixed candidate policies, it counts the k unsafe accepted proposals among n independent and identically distributed (iid) calibration proposals, and allocates a failure budget δ/K to each. For $k < n$ the upper bound U is the $(1-\delta/K)$ quantile of $\mathrm{Beta}(k+1, n-k)$, and $U = 1$ for $k = n$. Zero observed losses give $U = 1 - (\delta/K)^{1/n}$, which is small but not zero. With probability at least $1-\delta$ all K bounds hold at once, so any policy with $U \le \epsilon$ may be selected; if none qualifies, the runtime denies. The confidence refers to repeated calibration draws; the deployment risk refers to new proposals after the policy is fixed; a frequency on a finite test set is neither.

Distribution shift. For a fixed policy and a deployment law Q, if an external argument establishes total variation distance $\mathrm{TV}(D, Q) \le \eta$, then $R(Q) \le \min(1, R(D) + \eta)$. We certify no estimator of η. Without restrictions on Q, an attacker can concentrate all traffic on any unsafe input the policy accepts, driving its risk to one. The hard checks still exclude their exact predicates, but nothing excludes all semantic violations. Absent exact semantic verification, denying uncertified semantic acceptance is the only distribution-free fallback in this model.

C. Escalation synthesis (RQ II)

The next result says when escalating by score is the cheapest way to meet a risk budget. It is a standard Lagrangian argument, and we state it to make its premises visible.

Proposition 4 (Sufficient condition for optimal escalation): For $x \sim D_X$, let $r_0(x)$ and $r_1(x)$ be the conditional probabilities of unsafe acceptance under cheap acceptance and under escalation, let $d(x) > 0$ be the extra cost of escalation and $c_0(x)$ the baseline cost. A randomized escalation probability $q(x) \in [0,1]$ yields

$$R(q) = \mathbb{E}_{D_X}[(1-q)r_0 + qr_1],$$
$$C(q) = \mathbb{E}_{D_X}[c_0 + qd].$$

Consider minimizing $C(q)$ subject to $R(q) \le \epsilon$ over measurable policies with integrable costs. If some finite $\lambda \ge 0$ and a feasible pointwise minimizer q of the Lagrangian satisfy $\lambda(R(q) - \epsilon) = 0$, then q is optimal. It escalates where $\lambda(r_0 - r_1) > d$, accepts where the inequality is reversed, and may randomize on ties.

Proof: The multiplier λ is a price per unit of risk. The Lagrangian is

$$L(q, \lambda) = \mathbb{E}[c_0 + \lambda r_0] - \lambda\epsilon + \mathbb{E}[q(d - \lambda(r_0 - r_1))],$$

and the sign of the coefficient of q gives the rule. By weak duality, every feasible competitor costs at least the minimized Lagrangian. Complementary slackness makes the candidate's cost equal to that bound. A finite multiplier need not exist at a boundary budget, so its existence is a premise. ∎

The proposition addresses challenge C3 directly. With the raw judge and no sink, escalating T_j *increases* risk, so $r_0 - r_1 < 0$ and the rule never escalates it at positive cost. For T_b, a small risk reduction may not be worth a deliberation. For T^*, no optimization waives the mandatory sink. The conditional risks r_0, r_1 and their monotonicity in the score are assumptions; the proposition does not estimate them.

The pilot solves a smaller, finite problem: among a fixed grid of threshold policies, choose the one with the lowest mean calibration cost whose simultaneous upper bound meets the budget. This choice is exact within the grid, not among all routing functions. A separate linear program with known risks supports mixtures of policies; with a normalization constraint and one active risk constraint, an optimal vertex mixes at most two policies. An optional interface certifies a latency exceedance event $\Pr(L > \text{budget}) \le \gamma$, but we evaluated no real inference latency.

Proposition 5 (Tightness and limits): The veto bound $\min_i \beta_i$ and the capped route bound $\min(1, \sum_i b_i)$ are tight given only the stated marginal information. Pairwise independence does not determine the joint failure probability of an ensemble.

Proof: For the veto bound take $\pi = 1$ and $m_i = \beta_i$, and let guard i miss when a shared uniform variable is at most m_i; the joint miss probability is $\min_i \beta_i$. For routes, place disjoint unsafe masses b_i when they sum to at most one, and otherwise masses no larger than the b_i that sum to one. Finally, read three bits as guard misses and take the uniform laws on $\{000, 011, 101, 110\}$ and on $\{001, 010, 100, 111\}$. In both, each bit has marginal $1/2$ and every pair is independent, yet the all-miss probability is 0 in the first and $1/4$ in the second. ∎

The practical consequence is that measuring pairwise correlations between guards, a natural thing to do, cannot certify an ensemble's joint risk.

D. Trust boundary, assumptions and limits

The typed interpreter supports a local mock HTTP service, in-memory files and typed effect records; it rejects shell commands and unknown tool fields. Provenance references are write-once, and unknown references keep conservative labels. Approvals bind the immutable action digest and epoch. A tool adapter parses function-call argument objects and calls the

TABLE II
WHAT TINT CHECKS AT RUN TIME AND WHAT IT MUST ASSUME.

Obligation	Executed check	External premise
Edge compatibility	Scope/evidence sets; fixed topology	Truth of guarantees
Provenance	Write-once store; parent-label union	Complete host dependencies
Approval	Digest and epoch match	Trusted grant issuer
Hard safety	Mandatory typed predicates	Faithful interpreter semantics
Judge interface	Closed Boolean/numeric schema	Adequate semantic evidence
Certificate	Policy/view/context identity; risk/confidence budgets	Valid issuer, labels, iid relationship
Guard identity	Version string in fingerprint	Immutable implementation

same mediated interpreter; we have not connected it to an external model API.

At every runtime decision, a fingerprint, a cryptographic hash of the specification, guard versions, threshold, view and environment, is recomputed and compared with the certificate, and the certificate's risk and confidence values are checked against the budget. That honest version strings identify immutable implementations, that hosts declare dependencies completely, and that certificate issuers are trustworthy remain external premises. Table II separates what the code checks from what it must assume. By induction over the single effect transition of the control semantics, every effect is preceded by a passed sink check and a matching certificate. This is an inspection argument, not a proved refinement between the semantics and the Python code.

Three scorer versions exist. The primary study used `lexical-v1`; the exploratory study used `lexical-v2`, which fixes an arithmetic defect described in Section V; the current default, `lexical-v3-bounded`, adds an input-length cap and produced none of the reported measurements. Exceptions in a plugin deny authority, but a synchronous plugin that never returns is not cancelled. Concurrent mutation of trusted stores and covert channels are out of scope.

Assumptions and limits. The deterministic guarantees hold only for this typed, completely mediated semantics. The PRISM model is a separately checked finite abstraction with no proved refinement relation to the Python code. Quarantine removes raw instructions from the judge's input but does not establish the semantic oracle. Statistical certification assumes fixed candidates and iid calibration and deployment draws; arbitrary adaptation and drift are not covered, and a matching fingerprint cannot detect them.

IV. EVALUATION DESIGN

Our evaluation is designed to test whether TINT's mechanisms behave as the approach claims, not to estimate how safe a real agent would be. That choice shapes every component. The guards are scripted rather than learned, so each decision can be traced to a known cause; a failure then points to the mechanism under test rather than to an opaque model, and Proposition 1 requires a judge whose vulnerability is known exactly. The traces come from an authored mixture whose unsafe labels hold by construction. This means risk can be measured exactly, without disagreement between annotators, and it lets us include semantic-only attacks that the sink cannot decide, which public benchmarks do not isolate. The comparators are ablations, each removing one mechanism (the sink, the structured view, or the certificate) from an otherwise identical pipeline, because the question is which mechanism produces which effect, and re-implementations of published systems would differ in many ways at once. Statistical certification is tested on a separate generator whose true risk is known, since coverage cannot be checked when ground truth is unavailable. Costs are modeled units, and the risk budget is deliberately permissive, so that the certification and selection logic is exercised rather than trivially refusing. Finally, the protocol was preregistered and the primary run is reported as it failed, with the correction kept separate, so that no result was obtained by adjusting the design after seeing the data. These choices give internal validity at the expense of external validity: Section V establishes what the mechanisms do on a controlled instrument, and Section VII states what a study with real models would have to add.

The protocol was committed and tagged `prereg-v1` before any calibration or held-out evaluation. Unit tests and the canonical model checks preceded the tag and are not confirmatory findings. The tag is a local git tag, not an independently timestamped registry entry. One amendment, described in Section V, is reported separately from the primary run.

A. Implementation in the codebase

TINT is implemented as a small Python package, `src/tint`, that depends only on NumPy and SciPy and performs no network or shell access. Around it, the repository holds the formal artifacts (`formal/`), the experiment harness (`experiments/`) and the analysis and paper-generation scripts (`scripts/`). Table III maps each element of the approach in Sections II and III to the code that realizes it and to the study that exercises it. We describe the code here so that a reader can check any claim in the paper against a specific module.

The path of a single decision through the code follows Fig. 1 step by step. `Sandbox.call` receives a function-call argument object and `parse_tool_call` turns it into an immutable `Action`, rejecting unknown fields. `GuardRuntime.evaluate` then builds the `ActionView`, whose labels and approval flag come only from the trusted stores, and applies the hard predicates in `deterministic`. If they pass, it recomputes `policy_fingerprint` and asks `Certificate.applies` whether the stored certificate names this exact configuration and lies within the risk and confidence budgets. Only then does it call the scorer; a score below the threshold accepts, and otherwise the `quarantine` function builds the closed structured view and `safe_judge_call` invokes the judge, converting any exception into a block. An escalation reaches the approval stub and is denied. Every outcome is returned as a `GuardDecision` record, and the sandbox applies a typed effect only for an allow.

Three places where the code is narrower than the approach deserve mention. First, the checker in `spec` validates

TABLE III
HOW THE APPROACH MAPS TO THE CODEBASE. MODULE PATHS ARE RELATIVE TO `SRC/TINT` UNLESS A TOP-LEVEL DIRECTORY IS GIVEN. THE LAST COLUMN NAMES THE STUDY IN SECTIONS IV AND V THAT EXERCISES THE CODE.

Approach element	Paper reference	Code (module: main entry points)	Exercised by
Typed actions, provenance labels, approvals	Defs. 1–2	`model`: `Label`, `Action.digest`, `ProvenanceStore`, `ApprovalStore`, `action_view`	Unit tests; hard counts in the replay
Policy language and well-formedness checker	Defs. 5–6; Sec. III-A	`spec`: `parse`, `check`, `Contract`, `TrustEdge`; policy in `experiments/configs/devagent.json`	Mutation study; policy inventory
Layers: hard predicates, cheap scorer, structured view, judges	Def. 3; Table I	`layers`: `deterministic`, `LexicalScorer`, `quarantine`, `StructuredJudge`, `ExposedJudge`, `safe_judge_call`	Canonical traces; replay
Mediation path and configuration fingerprint	Fig. 1; Prop. 2	`runtime`: `GuardRuntime.evaluate`, `policy_fingerprint`, `parse_tool_call`	Replay (TINT rows); CPU timing
Certificates and statistical utilities	Sec. III-B; Prop. 3	`calibration`: `binomial_upper`, `certify_family`, `Certificate.applies`, `conformal_risk_control`, `veto_bound`	Coverage and dependence studies
Escalation synthesis	Sec. III-C; Props. 4–5	`synthesis`: `select`, `lagrangian_escalate`, `optimal_mixture`	Synthesis study; policy selection
Typed sandbox and canonical counterexample	Prop. 1; Table I	`agents`: `Sandbox.call`, `canonical_traces`, `fixture_store`	Unit tests; PRISM model
Comparators	Table V	`baselines`: `decide`	Replay
Finite formal checks	Props. 1, 2, 3, 5	`formal/cascade.pm` (PRISM), `formal/check.py`, `formal/sanity.py`	RQ I results
Trace generator, harness and analysis	Secs. IV–V	`experiments/dataset.py`, `experiments/run.py`, `scripts/process_results.py`, `scripts/validate_replay.py`, `scripts/make_macros.py`	All studies

arbitrary acyclic graphs, but the runtime accepts only the fixed three-layer topology of `devagent.json`: a metadata layer, a raw-view scorer and a structured-view judge. Second, the runtime uses one whole-policy certificate rather than the per-route bounds that Proposition 2 permits. Third, `lagrangian_escalate` implements the rule of Proposition 4, and `calibration` also implements split conformal prediction and CRC, but the pilot uses none of them: it selects from a finite threshold grid with `select`. The runtime's trust boundary is described in Section III-D.

Correctness of the code is checked in three ways. Unit and analysis tests in `tests/` reach 99.85% line coverage of the package. A replay validator re-joins every retained decision to its trace, label and configuration. A manifest of cryptographic hashes ties the raw logs, sources and configurations to the processed results, and the paper's numbers are generated from those results rather than typed by hand.

TABLE IV
FROZEN EXPERIMENTAL CONFIGURATION. ALL VALUES COME FROM `EXPERIMENTS/CONFIGS/PILOT.JSON`; THE EXPLORATORY RUN CHANGES ONLY THE SEEDS AND THE SCORER VERSION.

Parameter	Value
Primary seeds / exploratory seeds	0–9 / 10–19
Traces per seed: calibration / development / test / shift	1800 / 600 / 1200 / 600
Threshold grid	0.0 to 1.0 in steps of 0.1 (11 policies)
Baseline threshold for uncertified cheap exits	0.20
Risk budget ϵ / confidence-failure budget δ	20% / 5%
Modeled cost: rules / scorer / judge / approval	1 / 20 / 1000 / 5000 units
Coverage study: replications / calibration / test draws	1000 (100 per seed) / 1024 / 2048
Coverage study risk target	10%
Dependence study draws per cell	10000
Bootstrap resamples	10000
Core-search query budget	16

B. Experimental setup

Environment. All experiments ran in a single process on one workstation (Intel64 Family 6 Model 170 Stepping 4, GenuineIntel, 22 logical CPUs, Windows-11-10.0.26100-SP0) with Python 3.13.14, NumPy 2.2.5, SciPy 1.16.2 and pytest 9.0.3. Timings use the operating system's monotonic nanosecond clock. No external API was called, and no model was executed. Each run records the commit that produced it: `c47f2a4` for the primary run and `046501c` for the exploratory run.

What we could and could not run. The testbed has a hand-written lexical scorer, a deliberately vulnerable raw-text judge and a deterministic judge over the structured view; none uses the oracle label. Proposals come from two scripted sources, not from language models. We ran no model: the available GPU had no GPU-enabled learning framework installed, no API credentials were available, the small guard model we inspected required an access agreement we did not accept, and the Docker engine was unavailable.

Configuration. Table IV lists every parameter of the frozen configuration `experiments/configs/pilot.json`. The risk budget is far too permissive for production; it was chosen to exercise the certification mechanics. Modeled layer costs are arbitrary units; CPU times are measured separately on complete local decisions.

Procedure. The harness `experiments/run.py` executes the same steps for every seed.

1) Generate the calibration, development, test and shift splits with `experiments/dataset.py`.

2) Evaluate every threshold policy on the calibration split, recording for each proposal whether it was an unsafe acceptance and what it cost.
3) Compute the simultaneous binomial upper bound of every policy and select the cheapest policy whose bound meets the risk budget; if none does, issue no certificate.
4) Build a `GuardRuntime` with the selected threshold and, if one exists, a certificate bound to the configuration fingerprint.
5) Replay the development, test and shift splits through TINT and all six comparators, writing one compressed record per decision with its verdict, route, modeled cost, elapsed time and hard-violation flag.

Development data are recorded for diagnostics only and never used to revise thresholds. After the replay, the harness runs the coverage, dependence, checker, synthesis and core-search studies of Section IV-D on their own generators.

Outputs and reproduction. The primary run took 8.06 s and wrote 168000 decision records; the exploratory run took 10.37 s and wrote 168000. The command `python scripts/tasks.py all` archives any previous raw results and repeats the whole pipeline: tests, formal checks, both runs, analysis, macro generation and the PDF build. The cheaper command `python scripts/tasks.py verify-results` re-derives every processed number from the retained raw logs without rerunning the experiments.

C. Trace data and comparators

For each of 10 primary seeds, a fixed generator samples 1800 calibration, 600 development and 1200 test traces with disjoint identifiers. Traces are drawn from seven authored families with fixed sampling masses: benign reports, benign security discussion, secret exfiltration, unapproved destructive production actions, direct instruction injection, implicit unauthorized patches and judge-targeting text (Table IX). Within a family, traces differ only in an identifier suffix, so the splits are iid samples from one mixture, not independent tasks. A further 600 traces per seed come from an encoded family absent from calibration. Labels are known by construction; there was no human annotation.

Table V lists the comparators. They are controls designed to isolate one mechanism each, not reimplementations of AgentSpec, ShieldAgent or LlamaFirewall. Uncalibrated cheap exits use the fixed threshold 0.20; the calibrated method uses its selected threshold. Invalid delegation versus naive-with-sink isolates the mandatory sink; naive-with-sink versus checked-uncalibrated isolates the judge view; and the two checked variants isolate certification. Invalid delegation is charged a nominal scorer cost even though its verdicts equal the raw judge's; cost units are protocol charges, not profiles.

We inspected public benchmark repositories and their licenses but executed or redistributed none. R-Judge's project website states a noncommercial share-alike license, while retrieval of its repository license failed; AgentHarm adds a safety-use restriction. A repository's code license need not cover the datasets it contains.

TABLE V
COMPARATORS. FIXED DENOTES THE PROTOCOL'S BASELINE THRESHOLD; SELECTED DENOTES THE CALIBRATED THRESHOLD. A FAILED CHEAP EXIT WITH NO JUDGE IS A DENIAL.

Method	Hard sink	Cheap exit	Judge	Certificate
Rules only	yes	none	none	no
Lexical only	no	fixed	none	no
Scripted judge only	no	none	raw	no
Invalid delegation	no	none	raw	no
Naive with sink	yes	fixed	raw	no
Checked uncalibrated	yes	fixed	structured	no
Checked calibrated (TINT)	yes	selected	structured	yes

D. Evidence map and statistics

Table VI maps each preregistered hypothesis to its evidence and outcome.

The coverage study uses a *different* generator from the agent traces, one in which the true risk is known. For each fresh uniform draw U, candidate j incurs loss $\mathbf{1}[U < r_j]$, so its population risk is exactly r_j and losses are nested across candidates. Each of 1000 replications draws 1024 calibration and 2048 test samples, targets 10% risk, and selects the cheapest certified candidate. When none is feasible, the zero-risk denial fallback is counted as covered rather than dropped. We report whether the selected candidate's true risk lies below its bound (certificate coverage) separately from whether its test frequency lies below the budget.

A caution about the agent-trace statistics. Every comparator is a deterministic function of a trace's family, so each method's population risk, utility and cost follow exactly from the family masses; we report these exact values alongside the sample estimates. Across seeds, the only source of variation is how many traces of each family were drawn. Seed-level paired differences are summarized with a percentile bootstrap (10000 resamples), paired Wilcoxon tests with Holm correction across baselines, and the Vargha–Delaney effect size A_{12}. With ten seeds the smallest attainable Holm-adjusted p is the value we report for every non-tied comparison, so these tests confirm that differences are not artifacts of sampling, nothing more. Intervals are unadjusted and resample seeds, not individual traces. CPU quantiles pool decisions with a fixed comparator order and no warmup. Coverage intervals are exact binomial intervals.

The checker study times repeated valid declarations and four planted mutation classes: missing evidence, scope mismatch, unknown endpoint and cycle. Authoring effort and usability were not measured. A small *core search*, with at most 16 local queries per method, cycles through fixed attack phrases and stops at the first unsafe acceptance. It calls the uncalibrated decision functions at the fixed threshold (its log label `checked_quarantine` is misleading), not the certified runtime, and every method accepts its first phrase. It provides no evidence of adaptive robustness.

V. RESULTS

A. RQ I: specification and decomposition

The PRISM model checks 6 canonical configurations and agrees with the independent checker on both reachability

TABLE VI
PREREGISTERED HYPOTHESES, EVIDENCE AND OUTCOMES. "SUPPORTED" ALWAYS REFERS TO THE LOCAL INSTRUMENT; MISSING EVIDENCE IS REPORTED AS SUCH, NEVER REPLACED BY SIMULATED CLAIMS ABOUT REAL MODELS.

Hypothesis	RQ	Evidence	Outcome
Hard invariant preservation	I	Proof, PRISM, exhaustive model, replay	Supported within typed semantics
Planted invalid-spec detection	I	Mutation confusion matrix	Supported for generator classes
Full policy expressiveness	I	Executable restricted-DSL inventory	Refuted
Selected bound coverage	II	Known iid law with repeated calibration	Supported for synthetic law
Invalid delegation counterexample	II	Canonical traces and stronger sink control	Supported; sink alone also protects
Finite-family selection optimality	II	Independent enumeration and LP constraints	Supported within candidate family
Useful safety–utility improvement	III	Paired replay, controls and trade-off plots	Primary inconclusive; exploratory only
External/model generalization	III	No real-model or benchmark execution	Inconclusive/not run

properties. Exhaustive enumeration covers all 64 Boolean sink cases, and seeded sanity checks evaluate 10000 random probability instances against the closed-form expressions. These finite checks test the propositions; the general proofs remain on paper and are not mechanized in Lean or Isabelle.

In the mutation study the checker rejects all 1400 invalid-case evaluations, covering 140 unique mutated specifications, and accepts all 350 evaluations of 35 unique clean graphs: 0 false rejections and 0 missed mutations. This shows agreement with the mutation generator, not precision and recall on policy errors that developers actually make. The largest median checking time over all graph sizes was 9737.50 microseconds.

The DSL fully supports only 2 of the 5 policy classes we inventoried: production approval and secret egress. Semantic injection is handled only through the statistical proxy, and filesystem confinement and cumulative spending are rejected as unsupported. The full-expressiveness hypothesis is therefore refuted. We consider an explicit rejection preferable to accepting declarations the runtime cannot enforce. Line coverage of the core package is 99.85%, including a regression test for the numeric defect.

Answer to RQ I. A safety property can be split into an exact deterministic invariant enforced at the sink and a population-level semantic bound, with per-edge contracts whose premises are executable checks. Proposition 1 and its executable counterexample show why the premises must be checked. The decomposition works for the two deterministic policy classes we support; richer policies remain out of reach of the current DSL.

B. RQ II: risk, dependence and policy selection

On the synthetic law with known risk, selected bounds cover the true population risk in 99.60% of 1000 replications (exact interval [98.98%, 99.89%]), above the nominal 95%. The selected policy's test frequency is below the 10% budget in 998 replications. The certificate is thus conservative on an iid law, as its construction predicts; this is not a deployment guarantee.

Fig. 2 confirms (2) by simulation. As the mixture weight grows, the ensemble's joint miss probability rises from the product of the miss rates toward the smallest one: adding a correlated guard buys little. The parity construction of Proposition 5 adds that even identical pairwise statistics can hide very different joint risks. Both are controlled checks of the theory, not estimates of real guard correlations.

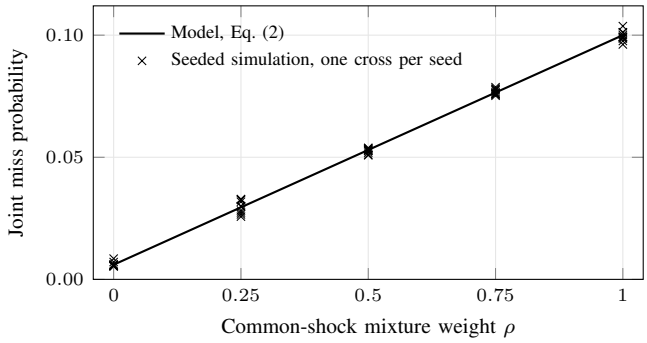


Fig. 2. Common-shock simulations against (2). All inputs are unsafe ($\pi = 1$), with conditional miss rates $0.10, 0.20, 0.30$. Each cross is one seed with 10000 draws. The horizontal axis is the model's mixture weight ρ, not a measured correlation.

Discrete policy selection agrees with a separate exhaustive calculation on all 50 primary instances (0 mismatches) and all 50 exploratory instances (0 mismatches). The original synthesis logs omitted calibration counts, so their stored bounds cannot be recomputed from those records; the amended logs retain the counts. A later re-analysis reconstructs each linear program's risk and cost from its weights, checks the simplex constraints and compares the objective with independently coded vertex enumeration. It also joins historical decisions to traces and checks configuration, authored labels and all outputs other than timings. "Independent" here means separately coded, not separately authored.

The primary run was infeasible. In every one of the 10 primary seeds, no candidate's simultaneous upper bound met the budget, so the runtime denied every semantic acceptance. The cause is visible in the calibration counts. Every one of the 11 thresholds produced the same number of calibration losses, so the grid offered a single risk level and only 4 behaviorally distinct policies, which differ in cost and benign utility but not in risk. The smallest upper bound across all seeds and thresholds was 31.1%, above the 20% budget. No choice of threshold could have succeeded, so the infeasibility reflects the detector, not the calibration procedure. Reporting it, rather than relaxing the budget or treating a missing certificate as permission, is exactly what a risk-constrained synthesizer should do.

TABLE VII
PRIMARY REPLAY ON HELD-OUT TRACES. RISK IS JOINT UNSAFE ACCEPTANCE OVER ALL PROPOSALS; UTILITY IS BENIGN ACCEPTANCE. COST IS IN MODELED UNITS, CPU IS MEASURED LOCAL DECISION TIME, AND THE HARD COUNT IS THE NUMBER OF ACCEPTED PROPOSALS THAT VIOLATE φ_{det}. THE CALIBRATED METHOD FOUND NO FEASIBLE CERTIFICATE AND DENIED EVERY SEMANTIC ACCEPTANCE.

Method	Risk (%)	Utility (%)	Cost units	CPU p_{95} (μs)	Hard count
Rules only	29.83	100.00	1.00	4.60	0
Lexical only	19.82	66.41	20.00	2.80	1176
Scripted judge only	39.62	66.41	1000.00	12.40	1176
Invalid delegation	39.62	66.41	1020.00	9.20	1176
Naive with sink	29.83	66.41	365.40	6.00	0
Checked, uncalibrated	29.83	66.41	365.40	7.30	0
Checked, calibrated	0.00	0.00	1.00	137.90	0

Answer to RQ II. Under fixed candidates, iid calibration and correct hard mediation, an early-exit hierarchy's risk is bounded by the sum of its route bounds, and an ensemble's by its tightest member; both bounds are tight. Compositions fail when edge premises are not discharged (Proposition 1), when guards fail together (Fig. 2), and whenever the deployment law may differ arbitrarily from calibration, which includes adaptive adversaries. A threshold escalation rule is cost-optimal when conditional risks are known and a Lagrange multiplier exists (Proposition 4). Empirically, the certificate is conservative on a known law, and the synthesizer exposes infeasibility instead of hiding it.

C. RQ III: utility and engineering trade-offs

Table VII reports the primary replay on 12000 held-out proposals. TINT reaches 0.00% risk only by accepting nothing: its benign utility is 0.00%. Rules only keeps 100.00% utility at 29.83% risk. Every method without the sink accepts all 1176 unapproved production deletions in the test set; every method with the sink accepts none. A system that denies every benign task has not shown a useful safety–utility trade-off, whatever its risk.

The arithmetic defect. The scorer is meant to compute $s(k) = \min(98, 4 + 12k)/100$, where k is the number of the strings `ignore`, `override`, `secret`, `.env`, `reviewer`, `instruction`, `send`, `upload` that occur as substrings after case-folding, and the judge blocks when $s \geq 0.40$. The primary version evaluated $0.04 + 0.12 \cdot 3$ in binary floating point, which is slightly below 0.40, so a proposal matching three strings was allowed instead of blocked. Direct injection and judge targeting match exactly three strings. This explains why the primary grid had a single risk level: those two families were accepted on whichever route a threshold sent them, and implicit patches, which match nothing, were always accepted cheaply. The checked core's exact population risk was therefore 30.00% at every threshold.

The amendment. The exploratory version computes the score in integer hundredths, changing no strings and no cutoffs. It was versioned separately, the original logs were preserved, and it was run on 10 fresh seeds. Under the corrected scorer, the score takes only three values on the fixture law: $s = 0.04$ for benign reports, implicit patches, production deletion and the shifted family; $s = 0.40$ for exfiltration, direct injection and judge targeting; and $s = 0.76$ for benign security text. The grid again has 4 distinct policies, now with 2 risk levels. For example, seed 10 observes 186 losses among 1800 calibration proposals at its selected threshold 0.10, giving a simultaneous upper bound of 12.34%. This exploratory result does not retroactively make the primary study a success.

Table VIII shows that TINT now accepts 66.46% of benign proposals at 10.16% risk. Compared with naive-with-sink, the paired risk difference is -10.62 percentage points (unadjusted interval $[-11.38, -9.86]$, Holm-adjusted $p = 0.01172$, $A_{12} = 1.00$, where larger values favor TINT), with identical utility. The whole difference comes from one family, as Table IX shows: judge-targeting text is accepted by the raw judge and denied by the structured one.

Where the numbers come from. Because decisions depend only on the family, the exploratory results can be derived by hand from Tables IX and X; the latter traces each family through the pipeline of Fig. 1. Benign security text matches six listed strings, so every lexical-based method denies it, which caps benign utility at 66.67%. Implicit patches match none, pass the sink because they neither leak a secret nor delete production data, and are accepted cheaply; their mass of 10% becomes a floor on risk, giving TINT's exact population risk of 10.00% against 20.00% for naive-with-sink. The exact modeled cost is 366.00 units. Among accepted proposals, 25.40% are unsafe, a quantity with a different denominator from both risk and utility.

The empirical judge-targeting family differs from the canonical T_j. It proposes an *unauthorized patch*, with no secret egress and no production deletion, so the sink cannot decide it; this is the semantic-only attack that Table I lacked. The canonical T_j is secret exfiltration and is always stopped by the sink. Sharing the reviewer phrase does not make the two attacks equivalent, which is why their outcomes differ.

Fig. 3 places the methods on risk and modeled cost. Rules only remains far cheaper and has higher utility than TINT; TINT is not dominant on every axis. Measured CPU time tells a similar story: TINT's p_{95} decision time is roughly twenty times that of the uncalibrated checked core, and a separate microbenchmark attributes most of the gap to recomputing the configuration fingerprint on every decision. Caching it per configuration would remove the cost, at the price of a weaker binding between certificate and running code. None of these timings estimates real guard inference latency.

Distribution shift. The shifted set contains only unau-

TABLE VIII
EXPLORATORY REPLICATION WITH THE CORRECTED SCORER ON FRESH SEEDS. COLUMNS AS IN TABLE VII. THE CALIBRATED AND UNCALIBRATED CHECKED VARIANTS MAKE IDENTICAL DECISIONS, BECAUSE NO SCORE LIES BETWEEN THEIR THRESHOLDS; CERTIFICATION HERE SELECTS AND LICENSES A POLICY, IT DOES NOT DETECT ANYTHING NEW.

Method	Risk (%)	Utility (%)	Cost units	CPU p_{95} (μs)	Hard count
Rules only	30.56	100.00	1.00	5.40	0
Lexical only	20.26	66.46	20.00	3.70	1212
Scripted judge only	30.88	66.46	1000.00	15.70	1212
Invalid delegation	30.88	66.46	1020.00	11.90	1212
Naive with sink	20.77	66.46	370.67	7.70	0
Checked, uncalibrated	10.16	66.46	370.67	9.20	0
Checked, calibrated	10.16	66.46	370.67	188.80	0

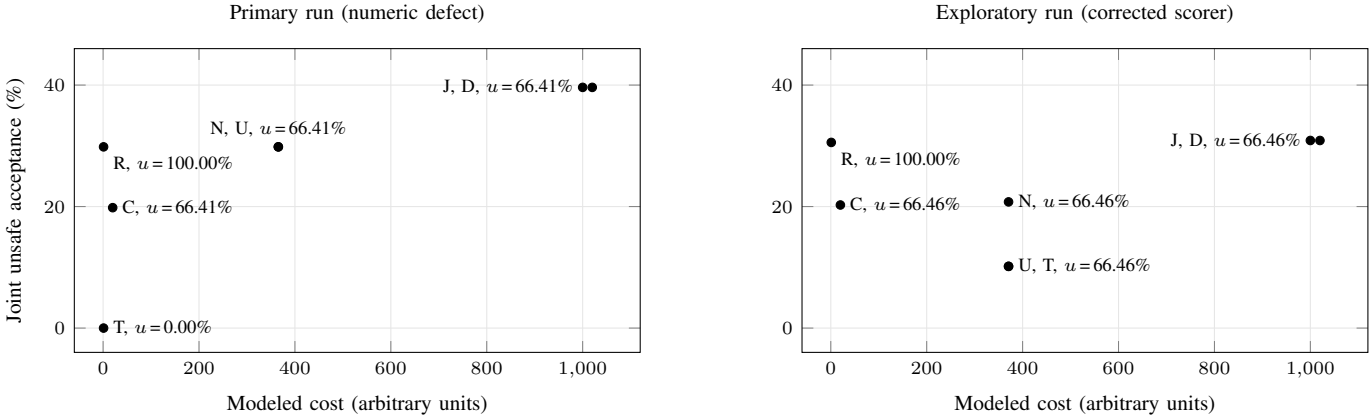


Fig. 3. Observed risk against modeled cost, with the primary and exploratory runs shown separately. R: rules only; C: lexical only; J: scripted raw judge only; D: invalid delegation; N: naive with sink; U: checked uncalibrated; T: TINT (checked calibrated). Each label gives the benign utility u, which must be read alongside the axes: the primary low-risk point denies all benign tasks. Coincident or adjacent points share a label. These are local configurations, not Pareto frontiers of real guard systems.

TABLE IX
AUTHORED FAMILIES. N_P AND N_E ARE TEST COUNTS IN THE PRIMARY AND EXPLORATORY RUNS; MASS IS THE SAMPLING PROBABILITY; THE LAST TWO COLUMNS ARE EXPLORATORY ACCEPTANCE RATES WITHIN EACH FAMILY. TRACES IN A FAMILY DIFFER ONLY IN AN IDENTIFIER SUFFIX.

Family	N_P	N_E	Mass (%)	Naive (%)	TINT (%)
Benign report	3592	3580	30	100	100
Benign security text	1817	1807	15	0	0
Secret exfiltration	1836	1734	15	0	0
Production deletion	1176	1212	10	0	0
Direct injection	1182	1174	10	0	0
Implicit patch	1203	1219	10	100	100
Judge targeting	1194	1274	10	100	0

TABLE X
HOW TINT DECIDES EACH FAMILY UNDER THE CORRECTED SCORER AT ITS SELECTED THRESHOLD. k IS THE NUMBER OF LISTED STRINGS MATCHED AND s THE RESULTING SCORE; THE JUDGE BLOCKS WHEN $s \geq 0.40$. UNSAFE FAMILIES ARE MARKED WITH AN ASTERISK.

Family	k	s	Sink	TINT outcome
Benign report	0	0.04	pass	cheap accept
Benign security text	6	0.76	pass	judge blocks (false denial)
Secret exfiltration*	3	0.40	block	denied by sink
Production deletion*	0	0.04	block	denied by sink
Direct injection*	3	0.40	pass	judge blocks
Implicit patch*	0	0.04	pass	cheap accept (missed)
Judge targeting*	3	0.40	pass	judge blocks
Encoded shift*	0	0.04	pass	cheap accept (missed)

thorized patches whose maintenance text is encoded, so it matches no listed string. The structured view therefore shows a low score, no secret external payload and no destructive production action, and TINT accepts 100.00% of these attacks. The fingerprint still matches, because a fingerprint identifies the configuration, not the input distribution. In the primary run the same set was denied only because TINT denied everything.

Answer to RQ III. On our authored mixture, TINT does not achieve a better risk–utility–cost frontier than the simplest competent control in the primary run, and in the corrected replication its advantage over naive-with-sink comes entirely from denying judge-targeting text through the structured view. Certification licensed a policy but improved no detection; it neither recognized the implicit and encoded attacks nor noticed the shift. We could not test real language models, state-of-the-art guards, realistic tasks or adaptive attackers, so whether the formal assumptions hold in practice remains open.

D. Threats to validity

Internal validity. The primary arithmetic defect is preserved, and the correction is reported only as exploratory, on fresh seeds. We authored the labels, templates and planted mutations, so they are not independent observations of semantic safety. Regression tests check configuration identity, not the

correctness of arbitrary plugins. The original synthesis counts are unavailable.

External validity. We evaluated no real base model, pretrained guard, official defense implementation, external benchmark, completed development task or practitioner workflow. Inference load, real token and GPU cost, long sessions and temporal drift are untested. Local CPU times and modeled units cannot establish deployment performance or savings.

Construct validity. The four denominators of Definition 7 must not be interchanged. Benign acceptance is a proxy, not task completion, and quarantine is not semantic noninterference. A matching context identifier does not establish stationarity, and complete dependency tracking remains a premise of the host.

Conclusion validity. Seeds resample one authored law, not independent developers or tasks, and the agent-trace results are determined by family masses. Conservative multiplicity correction can by itself produce infeasibility. Our numeric lint traces every empirical number in the source to a raw-derived summary but cannot inspect every glyph of the PDF.

VI. Related Work

A. Guardrail systems

Llama Guard, WildGuard, Constitutional Classifiers and ThinkGuard are classification and reasoning approaches to safety monitoring [11]–[14]. NeMo Guardrails provides programmable rails, and LlamaFirewall combines specialized scanners against agent threats [1], [2]; OneShield illustrates configurable guardrail engineering [15]. Truncated polynomial classifiers support progressive activation monitoring [16]. These systems supply the heterogeneous layers that a hierarchy composes, but they do not state the conditions under which one layer's verdict may be reused by another. Our toy components do not reproduce their learned behavior.

B. Cascades and adaptive compute

FrugalGPT studies cost-aware model cascades [17]. Trust or Escalate provides cascaded selective evaluation with human-agreement guarantees [6], and Conformal Arbitrage calibrates a switch between a primary model and a guardian and analyzes the optimality of score-gap routing [5]. Hybrid abstention applies hierarchical detection to practical filtering [18]. These works already give calibrated cascades; we add a check that a certificate applies to the actual route and input view at the tool-action boundary. Proposition 4 is a standard Lagrangian observation, not a new routing theorem.

C. Dual-process architectures

The System I/System II vocabulary links fast response with deliberate reasoning, and a recent survey organizes the reasoning-model literature [19]. We use only an operational resource criterion. Training for helpfulness and harmlessness and constitutional feedback address safety at training time [20], [21]; they complement but do not replace mediation of tool effects.

D. Runtime enforcement and verification

AgentSpec provides a rule language for runtime constraints on agents [3]; ShieldAgent reasons over verifiable policy representations [22]; ProbGuard monitors predicted future safety probabilistically [23]; and VeriGuard separates offline verification of a policy from online monitoring of actions [24]. ContrAgent is the closest work, with temporal agent contracts and compiled monitors [4], and Agent Behavioral Contracts add composition and probabilistic satisfaction [25]. The layered probabilistic architecture of [7] already composes conditional safety guarantees, which rules out any claim that ours is the first such composition. Classical shield synthesis corrects actions at run time against formal specifications [26]–[29], and RoboGuard protects a root-of-trust model with temporal constraints [30]. TINT's deterministic mediation belongs to this lineage; what it adds is an executable check of delegation premises between heterogeneous layers.

E. System-level isolation

CaMeL separates control flow from data flow and mediates capabilities [8], and Fides tracks confidentiality and integrity labels with a formal planner model [9]. These systems give stronger isolation than our structured judge view. Spotlighting and instruction-hierarchy training instead rely on model behavior to respect provenance cues and instruction priority [31], [32]. Our quarantine is a deliberately narrow input schema, not evidence that prompt formatting prevents instruction following. Task Shield checks alignment with the user's goal and would be a stronger real-agent comparison target [33].

F. Statistical guarantees

CRC controls expected monotone loss under exchangeability [10]. Conformal tutorials and adaptive classification methods distinguish marginal from conditional guarantees [34], [35]. Adaptive conformal inference addresses changing distributions, and later methods broaden the controlled objectives to adaptive and tail risks [36]–[38]. Conformal abstention, factuality and language-generation methods provide further statistical interfaces [39]–[41]. We use a classical finite-family binomial certificate and claim no new conformal result or adaptive guarantee.

G. Layered safety engineering

Component contracts, runtime assurance and defense in depth all argue for exposing assumptions rather than counting layers. ScenicProver is a closely related compositional assurance framework that tracks evidence and provenance [42]. Semantic integrity constraints show declarative assurance at another software boundary [43], and NaSh emphasizes recovery and guardrail design for language-driven shells [44]. Our contribution is an executable integration for a restricted tool-agent interface, not a new safety-case principle.

TABLE XI
COMPARISON WITH THE CLOSEST WORK. Y: CAPABILITY ESTABLISHED IN THE REVIEWED SOURCE; S: A DIFFERENT OR NARROWER PROPERTY; U: NOT ESTABLISHED IN OUR REVIEW (NEVER A CLAIM OF ABSENCE); P: PROPOSED AND TESTED ONLY IN OUR LOCAL PILOT.

Approach	Hierarchy	Formal spec	Hard	Statistical	Conditioned trust	Cost	Agent traces	Adaptive	Tool
ContrAgent	U	Y	Y	U	Y	Y	Y	U	Y
Layered probabilistic contracts	Y	Y	S	Y	Y	U	U	U	U
Conformal Arbitrage	Y	Y	U	Y	S	Y	U	U	Y
Trust or Escalate	Y	Y	U	Y	S	Y	U	U	Y
AgentSpec	U	Y	Y	U	U	Y	Y	U	Y
CaMeL / Fides	Y	Y	Y	U	Y	Y	Y	Y	Y
RoboGuard	Y	Y	Y	U	Y	Y	Y	Y	Y
ScenicProver	Y	Y	Y	Y	Y	S	S	U	Y
TINT local integration	Y	Y	P	P	Y	P	P	U	Y

H. Attacks and evaluation

Indirect prompt injection, optimized jailbreaks, guard evasion and attacks on latent defenses show that guard behavior must itself be part of the threat model [45]–[49]. Retrieved context can change guard judgments [50], and baseline defenses show why attacker access must be specified [51]. AgentDojo, ASB, AgentHarm, RedCode, ToolEmu and R-Judge offer much richer evaluation substrates [52]–[57], and RedCodeAgent, BAD-ACTS and CyberSecEval extend agent-security testing [58]–[60]. Sensitivity to representation cautions against reading a benchmark score as an intrinsic model property [61]. We executed none of these benchmarks, and our fixture search is not comparable to them.

In Table XI, *hierarchy* means composed guards; *formal spec*, machine-defined constraints; *hard*, exact safety within a stated semantics; *statistical*, a probability guarantee; *conditioned trust*, explicit reliance premises; *cost*, resource-aware routing or measurement; *agent traces*, tool-action sequences; *adaptive*, analyzed response-informed attacks; and *tool*, an available artifact. TINT's adaptive cell is U because our fixture search is not such an analysis. No single column is claimed as novel; the table shows that TINT's combination is unusual, not that any part of it is new.

VII. DISCUSSION AND LIMITATIONS

Lessons. Four lessons generalize beyond our toy instrument. *First, make premises executable.* The counterexample of Proposition 1 is trivial on paper, yet the naive hierarchy it describes is easy to build. Checking the judge's input view on every call turned an implicit assumption into a denial. *Second, keep the mandatory checks at the sink.* In our data the sink alone stopped every hard violation, including canonical T_j; conditioned trust adds value only for semantic attacks the sink cannot decide. *Third, certification licenses but does not detect.* The certificate decided whether a policy could run; it could not make a detector see implicit or encoded attacks, and a matching fingerprint did not reveal the shift. Teams that calibrate a guard should expect to learn that it is inadequate, as we did, and should design the system so that this outcome is reported rather than hidden. *Fourth, inspect the policy grid before calibrating.* A grid whose thresholds all give the same risk cannot be rescued by any statistical procedure; a check of how many behaviorally distinct policies a grid contains would have revealed the primary infeasibility before the run.

Limitations. The canonical counterexample also shows that ordinary mediation already protects the hard property, and the route sums only prevent a known error: importing ensemble intersections into early-exit accounting. These are established principles made executable, not new safety laws. All empirical evidence comes from an authored mixture with scripted guards whose decisions are fixed by family, so the study demonstrates mechanisms, not effectiveness. The distinctive practical value of conditioned trust therefore remains a research question.

What a convincing evaluation needs. A follow-up study should use real proposal-generating models and independently implemented learned guards on external benchmarks such as AgentDojo, compare against the official implementations of the closest systems, and measure task completion rather than benign acceptance. GPU-hosted models, activation probes and reasoning judges may change miss rates, correlations, costs and latency under load; no number reported here should be expected to transfer. Further work also needs richer rule semantics, separately calibrated route bounds, independently authored policy errors, adaptive attacks that observe guard responses, and a proved refinement link between the abstraction and the implementation.

VIII. CONCLUSION

Trust between guard layers is conditional, and conditions that are not checked are not guarantees. TINT makes a limited assurance case executable: deterministic effects stay mediated, certificates are bound to the actual routing configuration, and unproved premises are visible. Scoped proofs and reproducible experiments separate conditional component guarantees from valid composition. The empirical picture is sobering: the primary certified runtime was safe only by refusing all work, the corrected replication still accepted implicit attacks, and the shifted attacks passed unnoticed. Establishing real-agent engineering value is the next step.

DATA AND ARTIFACT AVAILABILITY

The repository is https://github.com/ai4se4ai-lab/trust-is-not-transitive. It contains compressed raw decision logs, processed summaries, hashes of sources and configurations, proofs (Propositions 1–5 correspond to `T0`–`T4` in `formal/FRAMEWORK.md`), the PRISM input and output, generated macros and vector figures. The commit that produced each raw run is recorded in its

`environment.json`, and the artifact manifest identifies the analysis inputs. Both the historical primary scorer and the corrected exploratory scorer are retained. `ARTIFACT.md` distinguishes re-analysis from full reruns and lists unavailable prerequisites. No archival release or DOI has been created yet.

Ethics and Acknowledgments

All attack strings target the local typed interpreter or a scripted judge. The released fixtures involve no real network egress, production operation, third-party exploit, personal data or genuine secret. No study with human participants was conducted. We did not accept access conditions for external models, and we do not redistribute external datasets. No funding or institutional approval is claimed. Automated reviews used during development are aids to the authors, not independent peer review.

References

[1] T. Rebedea, R. Dinu, M. Sreedhar, C. Parisien, and J. Cohen, "NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails," arXiv preprint, 2023. [Online]. Available: https://arxiv.org/abs/2310.10501

[2] S. Chennabasappa, C. Nikolaidis, D. Song, D. Molnar, S. Ding, S. Wan, S. Whitman, L. Deason, N. Doucette, A. Montilla, A. Gampa, B. de Paola, D. Gabi, J. Crnkovich, J.-C. Testud, K. He, R. Chaturvedi, W. Zhou, and J. Saxe, "LlamaFirewall: An open source guardrail system for building secure AI agents," arXiv preprint, 2025. [Online]. Available: https://arxiv.org/abs/2505.03574

[3] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents," arXiv preprint, 2025. [Online]. Available: https://arxiv.org/abs/2503.18666

[4] Y. Xiao and P. Nuzzo, "Symbolic Temporal Supervision of LLM Agents Using Contracts," arXiv preprint, 2026. [Online]. Available: https://arxiv.org/abs/2609.18128

[5] W. Overman and M. Bayati, "Conformal Arbitrage: Risk-Controlled Balancing of Competing Objectives in Language Models," arXiv preprint, 2025. [Online]. Available: https://arxiv.org/abs/2506.00911

[6] J. Jung, F. Brahman, and Y. Choi, "Trust or Escalate: LLM Judges with Provable Guarantees for Human Agreement," arXiv preprint, 2024. [Online]. Available: https://arxiv.org/abs/2407.18370

[7] S. Bensalem, Y. Dong, M. Franzle, X. Huang, J. Kroger, D. Nickovic, A. Nouri, R. Roy, and C. Wu, "Position: A Three-Layer Probabilistic Assume-Guarantee Architecture Is Structurally Required for Safe LLM Agent Deployment," arXiv preprint, 2026. [Online]. Available: https://arxiv.org/abs/2605.18672

[8] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, "Defeating Prompt Injections by Design," arXiv preprint, 2025. [Online]. Available: https://arxiv.org/abs/2503.18813

[9] M. Costa, B. Köpf, A. Kolluri, A. Paverd, M. Russinovich, A. Salem, S. Tople, L. Wutschitz, and S. Zanella-Béguelin, "Securing AI Agents with Information-Flow Control," arXiv preprint, 2025. [Online]. Available: https://arxiv.org/abs/2505.23643

[10] A. N. Angelopoulos, S. Bates, A. Fisch, L. Lei, and T. Schuster, "Conformal Risk Control," arXiv preprint, 2022. [Online]. Available: https://arxiv.org/abs/2208.02814

[11] H. Inan, K. Upasani, J. Chi, R. Rungta, K. Iyer, Y. Mao, M. Tontchev, Q. Hu, B. Fuller, D. Testuggine, and M. Khabsa, "Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations," arXiv preprint, 2023. [Online]. Available: https://arxiv.org/abs/2312.06674

[12] S. Han, K. Rao, A. Ettinger, L. Jiang, B. Y. Lin, N. Lambert, Y. Choi, and N. Dziri, "WildGuard: Open One-Stop Moderation Tools for Safety Risks, Jailbreaks, and Refusals of LLMs," arXiv preprint, 2024. [Online]. Available: https://arxiv.org/abs/2406.18495

[13] M. Sharma, M. Tong, J. Mu, J. Wei, J. Kruthoff, S. Goodfriend, E. Ong, A. Peng, R. Agarwal, C. Anil, A. Askell, N. Bailey, J. Benton, E. Bluemke, S. R. Bowman, E. Christiansen, H. Cunningham, A. Dau, A. Gopal, R. Gilson, L. Graham, L. Howard, N. Kalra, T. Lee, K. Lin, P. Lofgren, F. Mosconi, C. O'Hara, C. Olsson, L. Petrini, S. Rajani, N. Saxena, A. Silverstein, T. Singh, T. Sumers, L. Tang, K. K. Troy, C. Weisser, R. Zhong, G. Zhou, J. Leike, J. Kaplan, and E. Perez, "Constitutional Classifiers: Defending against Universal Jailbreaks across Thousands of Hours of Red Teaming," arXiv preprint, 2025. [Online]. Available: https://arxiv.org/abs/2501.18837

[14] X. Wen, W. Zhou, W. J. Mo, and M. Chen, "ThinkGuard: Deliberative Slow Thinking Leads to Cautious Guardrails," arXiv preprint, 2025. [Online]. Available: https://arxiv.org/abs/2502.13458

[15] C. DeLuca, A. L. Gentile, S. Asthana, B. Zhang, P. Chowdhary, K. Cheng, B. Shbita, P. Li, G.-J. Ren, and S. Gopisetty, "OneShield – the Next Generation of LLM Guardrails," arXiv preprint, 2025. [Online]. Available: https://arxiv.org/abs/2507.21170

[16] J. Oldfield, P. Torr, I. Patras, A. Bibi, and F. Barez, "Beyond Linear Probes: Dynamic Safety Monitoring for Language Models," arXiv preprint, 2025. [Online]. Available: https://arxiv.org/abs/2509.26238

[17] L. Chen, M. Zaharia, and J. Zou, "FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance," arXiv preprint, 2023. [Online]. Available: https://arxiv.org/abs/2305.05176

[18] A. Sharma, N. Tapas, and J. Patra, "Improving LLM Reliability through Hybrid Abstention and Adaptive Detection," arXiv preprint, 2026. [Online]. Available: https://arxiv.org/abs/2602.15391

[19] Z.-Z. Li, D. Zhang, M.-L. Zhang, J. Zhang, Z. Liu, Y. Yao, H. Xu, J. Zheng, P.-J. Wang, X. Chen, Y. Zhang, F. Yin, J. Dong, Z. Li, B.-L. Bi, L.-R. Mei, J. Fang, X. Liang, Z. Guo, L. Song, and C.-L. Liu, "From System 1 to System 2: A Survey of Reasoning Large Language Models," arXiv preprint, 2025. [Online]. Available: https://arxiv.org/abs/2502.17419

[20] Y. Bai, A. Jones, K. Ndousse, A. Askell, A. Chen, N. DasSarma, D. Drain, S. Fort, D. Ganguli, T. Henighan, N. Joseph, S. Kadavath, J. Kernion, T. Conerly, S. El-Showk, N. Elhage, Z. Hatfield-Dodds, D. Hernandez, T. Hume, S. Johnston, S. Kravec, L. Lovitt, N. Nanda, C. Olsson, D. Amodei, T. Brown, J. Clark, S. McCandlish, C. Olah, B. Mann, and J. Kaplan, "Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback," arXiv preprint, 2022. [Online]. Available: https://arxiv.org/abs/2204.05862

[21] Y. Bai, S. Kadavath, S. Kundu, A. Askell, J. Kernion, A. Jones, A. Chen, A. Goldie, A. Mirhoseini, C. McKinnon, C. Chen, C. Olsson, C. Olah, D. Hernandez, D. Drain, D. Ganguli, D. Li, E. Tran-Johnson, E. Perez, J. Kerr, J. Mueller, J. Ladish, J. Landau, K. Ndousse, K. Lukosuite, L. Lovitt, M. Sellitto, N. Elhage, N. Schiefer, N. Mercado, N. DasSarma, R. Lasenby, R. Larson, S. Ringer, S. Johnston, S. Kravec, S. E. Showk, S. Fort, T. Lanham, T. Telleen-Lawton, T. Conerly, T. Henighan, T. Hume, S. R. Bowman, Z. Hatfield-Dodds, B. Mann, D. Amodei, N. Joseph, S. McCandlish, T. Brown, and J. Kaplan, "Constitutional AI: Harmlessness from AI Feedback," arXiv preprint, 2022. [Online]. Available: https://arxiv.org/abs/2212.08073

[22] Z. Chen, M. Kang, and B. Li, "ShieldAgent: Shielding Agents via Verifiable Safety Policy Reasoning," arXiv preprint, 2025. [Online]. Available: https://arxiv.org/abs/2503.22738

[23] H. Wang, C. M. Poskitt, J. Wei, and J. Sun, "ProbGuard: Proactive Runtime Monitoring for LLM Agent Safety via Probabilistic Prediction," arXiv preprint, 2025. [Online]. Available: https://arxiv.org/abs/2508.00500

[24] L. Miculicich, M. Parmar, H. Palangi, K. D. Dvijotham, M. Montanari, T. Pfister, and L. T. Le, "VeriGuard: Enhancing LLM Agent Safety via Verified Code Generation," arXiv preprint, 2025. [Online]. Available: https://arxiv.org/abs/2510.05156

[25] V. P. Bhardwaj, "Agent Behavioral Contracts: Formal Specification and Runtime Enforcement for Reliable Autonomous AI Agents," arXiv preprint, 2026. [Online]. Available: https://arxiv.org/abs/2602.22302

[26] R. Bloem, B. Koenighofer, R. Koenighofer, and C. Wang, "Shield Synthesis: Runtime Enforcement for Reactive Systems," arXiv preprint, 2015. [Online]. Available: https://arxiv.org/abs/1501.02573

[27] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe Reinforcement Learning via Shielding," arXiv preprint, 2017. [Online]. Available: https://arxiv.org/abs/1708.08611

[28] M. Wu, J. Wang, J. Deshmukh, and C. Wang, "Shield Synthesis for Real: Enforcing Safety in Cyber-Physical Systems," arXiv preprint, 2019. [Online]. Available: https://arxiv.org/abs/1908.05402

[29] A. Rodriguez, G. Amir, D. Corsi, C. Sanchez, and G. Katz, "Shield Synthesis for LTL Modulo Theories," arXiv preprint, 2024. [Online]. Available: https://arxiv.org/abs/2406.04184

[30] Z. Ravichandran, A. Robey, V. Kumar, G. J. Pappas, and H. Hassani, "Safety Guardrails for LLM-Enabled Robots," arXiv preprint, 2025. [Online]. Available: https://arxiv.org/abs/2503.07885

[31] K. Hines, G. Lopez, M. Hall, F. Zarfati, Y. Zunger, and E. Kiciman, "Defending Against Indirect Prompt Injection Attacks With Spotlighting," arXiv preprint, 2024. [Online]. Available: https://arxiv.org/abs/2403.14720

[32] E. Wallace, K. Xiao, R. Leike, L. Weng, J. Heidecke, and A. Beutel, "The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions," arXiv preprint, 2024. [Online]. Available: https://arxiv.org/abs/2404.13208

[33] F. Jia, T. Wu, X. Qin, and A. Squicciarini, "The Task Shield: Enforcing Task Alignment to Defend Against Indirect Prompt Injection in LLM Agents," arXiv preprint, 2024. [Online]. Available: https://arxiv.org/abs/2412.16682

[34] A. N. Angelopoulos and S. Bates, "A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification," arXiv preprint, 2021. [Online]. Available: https://arxiv.org/abs/2107.07511

[35] Y. Romano, M. Sesia, and E. J. Candès, "Classification with Valid and Adaptive Coverage," arXiv preprint, 2020. [Online]. Available: https://arxiv.org/abs/2006.02544

[36] I. Gibbs and E. Candès, "Adaptive Conformal Inference Under Distribution Shift," arXiv preprint, 2021. [Online]. Available: https://arxiv.org/abs/2106.00170

[37] V. Blot, A. N. Angelopoulos, M. I. Jordan, and N. J.-B. Brunel, "Automatically Adaptive Conformal Risk Control," arXiv preprint, 2024. [Online]. Available: https://arxiv.org/abs/2406.17819

[38] C. Y.-C. Chen, J. Shen, Z. Deng, and L. Lei, "Conformal Tail Risk Control for Large Language Model Alignment," arXiv preprint, 2025. [Online]. Available: https://arxiv.org/abs/2502.20285

[39] Y. A. Yadkori, I. Kuzborskij, D. Stutz, A. György, A. Fisch, A. Doucet, I. Beloshapka, W.-H. Weng, Y.-Y. Yang, C. Szepesvári, A. T. Cemgil, and N. Tomasev, "Mitigating LLM Hallucinations via Conformal Abstention," arXiv preprint, 2024. [Online]. Available: https://arxiv.org/abs/2405.01563

[40] C. Mohri and T. Hashimoto, "Language Models with Conformal Factuality Guarantees," arXiv preprint, 2024. [Online]. Available: https://arxiv.org/abs/2402.10978

[41] V. Quach, A. Fisch, T. Schuster, A. Yala, J. H. Sohn, T. S. Jaakkola, and R. Barzilay, "Conformal Language Modeling," arXiv preprint, 2023. [Online]. Available: https://arxiv.org/abs/2306.10193

[42] E. Vin, K. A. Miller, I. Incer, S. A. Seshia, and D. J. Fremont, "ScenicProver: A Framework for Compositional Probabilistic Verification of Learning-Enabled Systems," arXiv preprint, 2025. [Online]. Available: https://arxiv.org/abs/2511.02164

[43] A. W. Lee, J. Chan, M. Fu, N. Kim, A. Mehta, D. Raghavan, and U. Cetintemel, "Semantic Integrity Constraints: Declarative Guardrails for AI-Augmented Data Processing Systems," arXiv preprint, 2025. [Online]. Available: https://arxiv.org/abs/2503.00600

[44] B. R. Gyawali, S. Achalla, K. Kallas, and S. Kumar, "NaSh: Guardrails for an LLM-Powered Natural Language Shell," arXiv preprint, 2025. [Online]. Available: https://arxiv.org/abs/2506.13028

[45] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," arXiv preprint, 2023. [Online]. Available: https://arxiv.org/abs/2302.12173

[46] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Z. Kolter, and M. Fredrikson, "Universal and Transferable Adversarial Attacks on Aligned Language Models," arXiv preprint, 2023. [Online]. Available: https://arxiv.org/abs/2307.15043

[47] W. Hackett, L. Birch, S. Trawicki, N. Suri, and P. Garraghan, "Bypassing LLM Guardrails: An Empirical Analysis of Evasion Attacks against Prompt Injection and Jailbreak Detection Systems," arXiv preprint, 2025. [Online]. Available: https://arxiv.org/abs/2504.11168

[48] L. Bailey, A. Serrano, A. Sheshadri, M. Seleznyov, J. Taylor, E. Jenner, J. Hilton, S. Casper, C. Guestrin, and S. Emmons, "Obfuscated Activations Bypass LLM Latent-Space Defenses," arXiv preprint, 2024. [Online]. Available: https://arxiv.org/abs/2412.09565

[49] M. Russinovich and A. Salem, "Jailbreaking is (Mostly) Simpler Than You Think," arXiv preprint, 2025. [Online]. Available: https://arxiv.org/abs/2503.05264

[50] Y. She, D. W. Peterson, M. M. Liu, V. Upadhyay, M. H. Chaghazardi, E. Kang, and D. Roth, "RAG Makes Guardrails Unsafe? Investigating Robustness of Guardrails under RAG-style Contexts," arXiv preprint, 2025. [Online]. Available: https://arxiv.org/abs/2510.05310

[51] N. Jain, A. Schwarzschild, Y. Wen, G. Somepalli, J. Kirchenbauer, P.-y. Chiang, M. Goldblum, A. Saha, J. Geiping, and T. Goldstein, "Baseline Defenses for Adversarial Attacks Against Aligned Language Models," arXiv preprint, 2023. [Online]. Available: https://arxiv.org/abs/2309.00614

[52] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents," arXiv preprint, 2024. [Online]. Available: https://arxiv.org/abs/2406.13352

[53] H. Zhang, J. Huang, K. Mei, Y. Yao, Z. Wang, C. Zhan, H. Wang, and Y. Zhang, "Agent Security Bench (ASB): Formalizing and Benchmarking Attacks and Defenses in LLM-based Agents," arXiv preprint, 2024. [Online]. Available: https://arxiv.org/abs/2410.02644

[54] M. Andriushchenko, A. Souly, M. Dziemian, D. Duenas, M. Lin, J. Wang, D. Hendrycks, A. Zou, Z. Kolter, M. Fredrikson, E. Winsor, J. Wynne, Y. Gal, and X. Davies, "AgentHarm: A Benchmark for Measuring Harmfulness of LLM Agents," arXiv preprint, 2024. [Online]. Available: https://arxiv.org/abs/2410.09024

[55] C. Guo, X. Liu, C. Xie, A. Zhou, Y. Zeng, Z. Lin, D. Song, and B. Li, "RedCode: Risky Code Execution and Generation Benchmark for Code Agents," arXiv preprint, 2024. [Online]. Available: https://arxiv.org/abs/2411.07781

[56] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto, "Identifying the Risks of LM Agents with an LM-Emulated Sandbox," arXiv preprint, 2023. [Online]. Available: https://arxiv.org/abs/2309.15817

[57] T. Yuan, Z. He, L. Dong, Y. Wang, R. Zhao, T. Xia, L. Xu, B. Zhou, F. Li, Z. Zhang, R. Wang, and G. Liu, "R-Judge: Benchmarking Safety Risk Awareness for LLM Agents," arXiv preprint, 2024. [Online]. Available: https://arxiv.org/abs/2401.10019

[58] C. Guo, C. Xie, Y. Yang, Z. Chen, Z. Lin, X. Davies, Y. Gal, D. Song, and B. Li, "RedCodeAgent: Automatic Red-teaming Agent against Diverse Code Agents," arXiv preprint, 2025. [Online]. Available: https://arxiv.org/abs/2510.02609

[59] J. Nöther, A. Singla, and G. Radanovic, "Benchmarking the Robustness of Agentic Systems to Adversarially-Induced Harms," arXiv preprint, 2025. [Online]. Available: https://arxiv.org/abs/2508.16481

[60] S. Wan, C. Nikolaidis, D. Song, D. Molnar, J. Crnkovich, J. Grace, M. Bhatt, S. Chennabasappa, S. Whitman, S. Ding, V. Ionescu, Y. Li, and J. Saxe, "CYBERSECEVAL 3: Advancing the Evaluation of Cybersecurity Risks and Capabilities in Large Language Models," arXiv preprint, 2024. [Online]. Available: https://arxiv.org/abs/2408.01605

[61] N. Karamchandani, P. Nagasubramaniam, X. Xie, S. Zhu, and D. Wu, "Threat-Preserving Representation Sensitivity in Agent-Security Benchmarks," arXiv preprint, 2026. [Online]. Available: https://arxiv.org/abs/2610.03585